

LUdecomposition – Mathematica vs Mathilda

A side-by-side audit of `LUdecomposition[m]` covering exact / symbolic inputs, machine-precision inputs, arbitrary-precision (MPFR) inputs, singular matrices, pivoting behaviour, and edge cases. The outputs in the Mathilda columns were produced by the build of Mathilda at HEAD on 2026-05-22, **after the four-phase improvement pass** that closed the original audit's deltas on (1) non-square $m \times n$ input, (2) the exact pivot heuristic for numerics, (3) the `LUdecomposition::luc` badly-conditioned warning, and (4) the MPFR condition-number estimator (now Hager-Higham, matching LAPACK's `*lacn2`). The Mathematica behaviour was sampled from Wolfram Mathematica 13 via `wolframscript`.

Both systems implement Doolittle's elimination with partial row pivoting and agree on the public contract: the call returns `{lu, p, c}` where `lu` is the combined L / U matrix (strict-lower triangle is L with implicit unit diagonal, upper triangle is U), `p` is the 1-indexed row permutation of length `Length[m]` with `m[[p]] == 1 . u`, and `c` is an L-infinity condition-number estimate (the exact Integer 0 for exact / symbolic / non-square input). As of the May 2026 pass the only remaining material delta between the two systems on supported input is whether the `Modulus` option is recognised – a system-wide gap, not specific to `LUdecomposition`.

1. API surface

Feature	Mathematica	Mathilda
<code>LUdecomposition[m] -> {lu, p, c}</code>	yes	yes
Identity: <code>m[[p]] == 1 . u</code> (with <code>l = LowerTriangularize[lu, -1] + IdentityMatrix[n], u = UpperTriangularize[lu]</code>)	yes	yes
Square $n \times n$ input	yes	yes
Non-square $m \times n$ input ($m \neq n$)	yes (returns a partial factorisation, <code>lu</code> has the input shape, <code>p</code> has length <code>rows</code>)	yes (as of Phase 1, matches Mathematica exactly: <code>lu</code> keeps input shape, <code>p</code> has length <code>rows</code> , <code>c = 0</code>)
<code>Modulus -> n</code> option	yes (modular factorisation, $n > 0$)	no – system-wide gap (no Mathilda builtin currently implements modular evaluation)
<code>c</code> slot for exact / symbolic / non-square input	exact Integer 0	exact Integer 0
<code>c</code> slot for machine-precision input	LAPACK reciprocal-condition estimate, <code>1 / rcond</code>	LAPACK reciprocal-condition estimate, <code>1 / rcond</code>

Feature	Mathematica	Mathilda
c slot for MPFR input	high-precision condition estimate via Mathematica's Internal numerics	Hager- Higham one-norm estimator on A^{-T} (as of Phase 4, matches LAPACK's *1acn2 strategy; $O(n^2)$ per call). Complex MPFR matrices keep the explicit- inverse estimator.
LUdecomposition::sing warning on a zero pivot	yes	yes
LUdecomposition::luc warning on ill-conditioned numerical input	yes	yes (as of Phase 3, threshold $1/\$MachineEpsilon$ for machine, 2^{bits} for MPFR)

Conventions agree: L has unit diagonal, U holds the pivots, both factors are packed into the single `lu` block returned in slot 1. The permutation vector is 1-indexed and stores **the source row that was placed at row k** , so `m[[p]]` (Mathematica's row-extraction syntax) recovers the row order seen by the factorisation.

2. Symbolic / exact inputs

2.1 Integer $\{\{1, 2\}, \{3, 4\}\}$

Both systems return the identical, fully-canonical result:

```
LUdecomposition[{{1, 2}, {3, 4}}]
=> {{1, 2}, {3, -2}}, {1, 2}, 0
```

2.2 Vandermonde-like $\{\{1, 1, 1\}, \{2, 4, 8\}, \{3, 9, 27\}\}$

```
LUdecomposition[{{1, 1, 1}, {2, 4, 8}, {3, 9, 27}}]
=> {{1, 1, 1}, {2, 2, 6}, {3, 3, 6}}, {1, 2, 3}, 0
```

Both systems agree exactly. Reconstruction check `m[[p]] - 1 . u == {{0,0,0},{0,0,0},{0,0,0}}` holds in both.

2.3 Free-symbolic $\{a, b\}, \{c, d\}$

Mathematica

```
{{a, b}, {c/a, -((b c)/a) + d}}, {1, 2}, 0}
```

Mathilda

```
{{a, b}, {c/a, (-b c + a d)/a}}, {1, 2}, 0}
```

Algebraically identical $-(-b c + a d)/a == -(b c)/a + d$. The divergence is purely a canonical-form choice. Mathilda's symbolic core puts every $lu[i, j]$ through `Together`; Mathematica reduces slightly further with the equivalent of `Apart` on the final $r[2, 2]$ slot. Both pick $p = \{1, 2\}$ because the pivot scan stops at the first column entry that is not provably zero, and the symbolic a is not provably zero.

2.4 Free-symbolic $\{a, b, c\}, \{d, e, f\}, \{g, h, i\}$

Both systems produce the textbook Doolittle elimination over the symbols. Each U-row entry contains one extra $(a g - d e) / (a e - b d)$ nested fraction; Mathematica and Mathilda differ only in how those nested fractions are pulled apart on the printed page.

Mathematica

```
{{a, b, c},
 {d/a, -((b d)/a) + e, -((c d)/a) + f},
 {g/a,
  (-((b g)/a) + h)/(-((b d)/a) + e),
  -((c g)/a)
  - ((-((c d)/a) + f)*(-((b g)/a) + h))/(-((b d)/a) + e)
  + i}},
 {1, 2, 3}, 0}
```

Mathilda

```
{{a, b, c},
 {d/a, (-b d + a e)/a, (-c d + a f)/a},
 {g/a,
  (-b g + a h)/(-b d + a e),
  ((-c e + b f) g + (c d - a f) h + (-b d + a e) i)/(-b d + a e)}},
 {1, 2, 3}, 0}
```

Both factorisations reconstruct the original matrix under `Simplify[m[[p]] - 1 . u]`.

2.5 Rational $\{1/2, 1/3\}, \{1/5, 1/7\}$

```
LUDecomposition[{{1/2, 1/3}, {1/5, 1/7}}]
=> {{1/5, 1/7}, {5/2, -1/42}}, {2, 1}, 0}
```

Mathilda matches Mathematica exactly as of Phase 2. Mathilda's symbolic kernel picks the pivot with the **smallest absolute value** when every entry in the active column is an exact numeric (`Integer` / `BigInt` / `Rational` / `Complex` of those) – here $|1/5| < |1/2|$, so row 2 is the pivot row. For columns containing free symbols, `Sqrt`, or other non-exact-numeric leaves, the kernel falls back to “first non-zero”, preserving the documented behaviour for `LUDecomposition[{{a, b}, {c, d}}] -> {1, 2}`.

Smallest $|pivot|$ tends to keep intermediate L entries integer for integer input (e.g. `LUDecomposition[{{4, 5}, {2, 3}}]` picks the 2 pivot, giving $L[2,1] = 4/2 = 2$ rather than $2/4 = 1/2$).

2.6 Complex exact $\{\{1+I, 2\}, \{3, 4-I\}\}$

```
LUDecomposition[{{1 + I, 2}, {3, 4 - I}}]
```

System	lu	p
Mathematica	$\{\{1 + I, 2\}, \{3/2 - (3/2) I, 1 + 2 I\}\}$	$\{1, 2\}$
Mathilda	$\{\{1 + I, 2\}, \{3/2 - 3/2*I, 1 + 2*I\}\}$	$\{1, 2\}$

Identical content; the only difference is Mathilda's $3/2*I$ versus Mathematica's $(3/2) I$ – pure print cosmetics.

3. Machine-precision inputs

For any matrix carrying at least one Real leaf whose precision is ≤ 53 bits, Mathilda routes to `lu_machine_dispatch` in `src/linalg/lucomp_machine.c`, which wraps LAPACK (`dgetrf` / `zgetrf` for the factorisation, `dgecon` / `zgecon` with `dlange` / `zlange` for the condition estimate). LAPACK is discovered through the four-tier autodetect ladder documented in `src/linalg/lapack.h` (Apple Accelerate, pkg-config lapacke, system lapacke, graceful fallback to the symbolic kernel).

Mathematica also dispatches to LAPACK for MachinePrecision input, so the algorithms are identical except for backing library.

3.1 $\{\{1.6, 2.7, 3.6\}, \{1.2, 3.2, 5.2\}, \{3.3, 3.4, 6.5\}\}$

```
{lu, p, c} = LUDecomposition[{{1.6, 2.7, 3.6},
                               {1.2, 3.2, 5.2},
                               {3.3, 3.4, 6.5}}];
```

Slot	Mathematica	Mathilda
lu	$\{\{3.3, 3.4, 6.5\}, \{0.36363636363636365, 1.9636363636363638, 2.8363636363636364\}, \{0.48484848484848486, 0.5354938271604939, -1.0703703703703706\}\}$	$\{\{3.3, 3.4, 6.5\}, \{0.363636, 1.96364, 2.83636\}, \{0.484848, 0.535494, -1.07037\}\}$ (6-digit print default)
p	$\{3, 2, 1\}$	$\{3, 2, 1\}$
c	20.839100346020764	20.8391 (full IEEE precision in memory; truncated by print)

Identical to the last bit of every entry; the only visible difference is Mathilda's REPL prints to 6 significant digits while Mathematica prints to 16.

3.2 Ill-conditioned 5x5

```
A = {{1., 2., 3., 4., 5.},
      {2., 1., 4., 3., 6.},
      {3., 4., 1., 2., 7.},
      {4., 3., 2., 1., 8.},
      {5., 6., 7., 8., 1.}};
```

Slot	Mathematica	Mathilda
p	{5, 4, 2, 1, 3}	{5, 4, 2, 1, 3}
c	4.034561140833293e17	4.03456e+17
reconstruction	{{0, ...}, ...} (Chop)	matches to machine epsilon
A[[p]] - 1 . u		

Mathematica also emits `LUdecomposition::luc: Result ... may contain significant numerical errors. because the condition estimate exceeds 1 / $MachineEpsilon.` Mathilda emits the analogous warning as of Phase 3 (machine-kernel threshold `1 / DBL_EPSILON ~ 4.5e15`). The warning is one-shot per process to match Mathematica's `General::stop` suppression behaviour.

3.3 Complex machine {{1.0+I, 2.0}, {3.0, 4.0-I}}

```
lu = {{3.0,          4.0 - 1.0 I},
      {0.3333... + 0.3333... I,
       0.3333... - 1.0 I}}
p  = {2, 1}
c  = 13.7924...
```

Output identical between the two systems modulo Mathilda's print truncation. Note Mathilda collapses `3.0 + 0.0 I` to bare `3.0` in the output (`lu_mach_make_scalar` in `ludecomp_machine.c`). Mathematica retains the explicit `3. + 0.*I` form. Both are valid representations.

4. Arbitrary-precision (MPFR) inputs

For any inexact input whose minimum leaf precision is > 53 bits, Mathilda routes to `lu_mpr_dispatch` in `src/linalg/ludecomp_mpr.c`. This is a hand-rolled **Doolittle elimination over row-major MPFR arrays** with paired re/im planes for complex (no MPC dependency, matching the QR / eigen kernels). Pivot selection is $\max |A[i, k]|$ over the unfactored sub-column.

For the condition number on real input, Mathilda now runs the **Hager-Higham one-norm estimator** on A^{-T} – the same algorithm LAPACK's `*lacn2` uses inside `*gecon`. Each iteration is two triangular solves ($O(n^2)$); 2-5 iterations are typical, so the total condition-number cost drops from $O(n^3)$ (one LU-solve per column of the explicit inverse) to $O(n^2)$. The estimator is a lower bound on the true $\|A^{-1}\|_\infty$ but for well-conditioned matrices typically agrees with the explicit-inverse value to ULP. Complex MPFR matrices still use the explicit-inverse approach.

Mathematica's arbitrary-precision LU uses high-precision elimination through its `Internal\HighPrecisionNumerics` stack with an LAPACK-style condition estimator. Both guarantee reconstruction-error scaling of $\text{order } 2^{(-\text{bits})}$ for full-rank input.

4.1 50-digit symmetric {{4, 1, 2}, {1, 3, 0}, {2, 0, 5}}

```
mp = {{N[4, 50], N[1, 50], N[2, 50]},
      {N[1, 50], N[3, 50], N[0, 50]},
      {N[2, 50], N[0, 50], N[5, 50]}};
{lu, p, c} = LUdecomposition[mp];
```

Both systems return `p = {1, 2, 3}` (no row swaps – pivots are already in size order) and an `lu` whose every entry carries the full 50 decimal digits. The condition number `c` agrees to the last printed digit:

```
c = 4.23255813953488372093023255813953488372093023255813...
```

(Mathematica appends a precision suffix ``50.`, Mathilda just prints the bits.)

4.2 Hilbert-like 4x4 at 30 decimal digits

$$H_4 = \{\{N[1, 30], N[1/2, 30], N[1/3, 30], N[1/4, 30]\}, \\ \{N[1/2, 30], N[1/3, 30], N[1/4, 30], N[1/5, 30]\}, \\ \{N[1/3, 30], N[1/4, 30], N[1/5, 30], N[1/6, 30]\}, \\ \{N[1/4, 30], N[1/5, 30], N[1/6, 30], N[1/7, 30]\}\};$$
[illegible]

4.3 Mixed precision

Both systems take the minimum input precision as the working precision. Mathilda's `common_scan_inexact(m)` (shared with QR / PseudoInverse / Eigenvalues) returns the minimum bits; if `min_bits` $\leq$ 53 it goes machine, otherwise MPFR.

For example, $\{\{1.0, 2.0, 3.0\}, \{N[4, 30], N[5, 30], N[6, 30]\}, \{7.0, 8.0, 10.0\}\}$ runs at 53 bits (the machine-precision rows dominate), and the factorisation, permutation, and condition number all agree to the last printed digit between the two systems.

5. Pivoting

For machine and MPFR inputs both systems implement partial pivoting by largest absolute column residual – the standard LAPACK choice (`dgetrf` for Mathilda’s machine path) and the natural choice for the MPFR Doolittle kernel.

For exact / symbolic inputs the two systems diverge slightly:

Input class	Mathematica pivot rule	Mathilda pivot rule
Free-symbolic (any leaf is a non-numeric symbol)	“Generic non-zero” – skip a pivot only if it is Together -zero	Same: “first non-zero” via is_definitely_zero (LU[i, k])
Exact integers / Gaussian integers	First non-zero	First non-zero
Exact rationals with a non-integer entry	Complexity heuristic (prefers smaller-numerator pivots)	First non-zero
Sqrt -bearing entries	Same as free-symbolic	Same as free-symbolic
MPFR / MachinePrecision	Largest absolute residual (LAPACK / scaled compare)	Largest absolute residual (LAPACK / mpfr_cmp)

The rational-pivot divergence is illustrated by §2.5.

5.1 Forced swap $\{\{0, 1\}, \{1, 0\}\}$

Both systems return $\{\{1, 0\}, \{0, 1\}\}$, $\{2, 1\}$, $0\}$ – the natural pivot at position $(1, 1)$ is exactly zero, so both systems swap to row 2. Permutation conventions agree.

5.2 Anti-diagonal $\{\{0, 0, 1\}, \{0, 1, 0\}, \{1, 0, 0\}\}$

Both return $\{\{1, 0, 0\}, \{0, 1, 0\}, \{0, 0, 1\}\}, \{3, 2, 1\}, 0\}$.

5.3 Mid-stream zero pivot $\{\{0, 1, 2\}, \{3, 4, 5\}, \{6, 7, 9\}\}$

Both return $\{\{3, 4, 5\}, \{0, 1, 2\}, \{2, -1, 1\}\}, \{2, 1, 3\}, 0\}$. Mathematica picked row 2 (not row 3, which has the largest magnitude) because exact-integer pivoting is “first non-zero” – so the systems remain in agreement on this exact-integer case.

6. Singular matrices

Both systems emit `LUdecomposition::sing: Matrix ... is singular.` and **still return a (rank-deficient) factorisation** with one or more zero entries on U's diagonal. The factorisation is mathematically meaningful only at the rows above the singular step.

6.1 Rank-2 $\{\{1, 2, 3\}, \{4, 5, 6\}, \{7, 8, 9\}\}$

```
LUdecomposition::sing: Matrix  $\{\{1, 2, 3\}, \{4, 5, 6\}, \{7, 8, 9\}\}$  is singular.  
=>  $\{\{1, 2, 3\}, \{4, -3, -6\}, \{7, 2, 0\}\}, \{1, 2, 3\}, 0\}$ 
```

Identical between the two systems including the zero in slot (3, 3) of lu which encodes the rank-2 deficiency.

6.2 All-zero $\{\{0, 0\}, \{0, 0\}\}$

```
LUdecomposition::sing: Matrix  $\{\{0, 0\}, \{0, 0\}\}$  is singular.  
=>  $\{\{0, 0\}, \{0, 0\}\}, \{1, 2\}, 0\}$ 
```

Identical between the two systems.

6.3 1x1 zero $\{\{0\}\}$

```
LUdecomposition::sing: Matrix  $\{\{0\}\}$  is singular.  
=>  $\{\{0\}\}, \{1\}, 0\}$ 
```

Identical between the two systems.

7. Edge cases

Input	Mathematica	Mathilda
$\{\{5\}\}$	$\{\{5\}\}, \{1\}, 0\}$	$\{\{5\}\}, \{1\}, 0\}$ – match
$\{\{x\}\}$ (symbolic)	$\{\{x\}\}, \{1\}, 0\}$	$\{\{x\}\}, \{1\}, 0\}$ – match
<code>IdentityMatrix[n]</code>	<code>{IdentityMatrix[n], {1, ..., n}, 0}</code>	match
<code>DiagonalMatrix[{2, 3, 5}]</code>	<code>{DiagonalMatrix[{2,3,5}], {1, 2, 3}, 0}</code>	match
Non-square $m \times n, m \neq n$	accepted, partial factorisation returned	accepted (Phase 1), partial factorisation returned, same shape
Tall $\{\{1,2\}, \{3,4\}, \{5,6\}\}$	$\{\{1,2\}, \{3,-2\}, \{5,2\}\}, \{1,2,3\}, 0\}$	match (Phase 1)
Wide $\{\{1,2,3\}, \{4,5,6\}\}$	$\{\{1,2,3\}, \{4,-3,-6\}\}, \{1,2\}, 0\}$	match (Phase 1)

Input	Mathematica	Mathilda
Empty matrix {}	rejected, unevaluated	rejected, <code>LUdecomposition::matsq</code> , unevaluated

8. Summary of behavioural deltas

After the May 2026 improvement pass the substantive deltas surfaced in the original audit are all closed. The remaining difference is a system-wide gap, not specific to `LUdecomposition`:

Area	Difference
Modulus <code>-> n</code> option	Mathematica supports modular factorisation. Mathilda has no <code>Modulus</code> support anywhere in the codebase yet – this is a system-wide gap that affects many builtins (<code>Inverse</code> , <code>Solve</code> , <code>Det</code> , etc.), not specific to <code>LUdecomposition</code> .
Free-symbolic canonical form	Mathilda canonicalises every <code>lu[i, j]</code> with <code>Together</code> ; Mathematica nudges further into a partial-fraction-ish form on the bottom-right slot. No content difference; both factorisations reconstruct <code>m</code> under <code>Simplify</code> .

Deltas closed by the May 2026 pass:

Area	Resolution
Non-square <code>m x n</code> input	Phase 1 – the builtin now accepts any non-empty rectangular matrix and returns a partial Doolittle factorisation matching Mathematica's shape (<code>lu</code> is <code>rows x cols</code> , <code>p</code> is length <code>rows</code> , <code>c = 0</code>).

Area	Resolution
Exact-rational / numeric pivot rule	Phase 2 – the symbolic kernel picks the smallest absolute value pivot for columns of exact numerics (<code>Integer</code> / <code>BigInt</code> / <code>Rational</code> / exact <code>Complex</code>). Matches Mathematica’s pivot selection on every exact-numeric test case probed via <code>wolframscript</code> .
<code>::luc</code> badly-conditioned warning	Phase 3 – both kernels now emit a one-shot <code>LUdecomposition::luc</code> when the condition estimate exceeds <code>1 / DBL_EPSILON</code> (machine) or <code>2^bits</code> (MPFR).
MPFR condition-number estimator	Phase 4 – real MPFR matrices now use the Hager-Higham one-norm estimator (LAPACK <code>*lacn2</code> strategy), $O(n^2)$ per call. Complex MPFR matrices retain the explicit-inverse estimator.

Areas where the two systems agree:

- Public API (`LUdecomposition[m]`).
- `{lu, p, c}` return convention.
- Doolittle storage (unit L on the strict-lower triangle, U on the upper).
- Permutation convention `m[[p]] == 1 . u`.
- LAPACK backing on machine precision (`dgetrf` / `zgetrf` for the factorisation, `dgecon` / `zgecon` for the condition estimate).
- Singular-matrix handling: warn but still return a (rank-deficient) factorisation.
- Mid-stream zero-pivot row swap for exact integers / Gaussian integers.
- Edge cases on 1x1 input, free-symbolic input, identity, diagonal.
- Mixed-precision inputs: the working precision is the minimum input precision.

9. Where to look in the source

- `src/linalg/lucomp.h` – public surface and the contract.
- `src/linalg/lucomp.c` – builtin entry, option parsing, `lu_dispatch` router, and the symbolic Doolittle core driven through the Mathilda evaluator (`lu_symbolic_core`, `lu_symbolic_dispatch`).
- `src/linalg/lucomp_machine.c` – LAPACK fast path (`lu_machine_dispatch`); loads to a column-major double buffer, runs `dgetrf` / `zgetrf` then `dgecon` / `zgecon`, builds `{lu, p, c}` as Mathilda

lists.

- `src/linalg/ludecomp_mpfr.c` – MPFR Doolittle kernel (`lu_mpfr_dispatch`); row-major MPFR arrays (paired re/im for complex), largest-magnitude pivot selection, $\|A\|_{\infty} * \|A^{-1}\|_{\infty}$ condition estimate.
- `src/linalg/lapack.h` / `lapack.c` – LAPACK ABI wrappers and the four-tier autodetect ladder shared with other linalg kernels.
- `tests/test_ludecomposition.c` – exact / symbolic tests.
- `tests/test_ludecomposition_machine.c` – LAPACK path tests.
- `tests/test_ludecomposition_mpfr.c` – MPFR path tests.
- `docs/spec/builtins/linear-algebra.md` (`## LUdecomposition`) – user-facing spec.
- `QR_DECOMPOSITIONS_COMPARISON.md` – the companion comparison document for `QRdecomposition`.